# TP: Cryptography, Digital Certificates, and PGP

## Table of Contents



## Initial notes

1. The concepts presented in this working document, as well as the proposed exercises, complement the material provided in the theoretical module and should not be used without a clear understanding of that module. **For the purpose of results presentation, you should record your observations in a *logbook*, at least including the tasks marked in red**. Final evaluation will be based on the logbook content and so it must be clear and objective.
2. Execution of the work proposed here require only one personal computer with Internet connection, an implementation of the OpenPGP, like PGP[1] (or GnuPG, or even the GPG4Win), a X509 certificate manager (included in most Operating Systems, or applications like Firefox browser, or the e-mail client Thunderbird) and an email client installed. Software installation is a fairly simple task and is not detailed in this document. Note: there is a great similarity, at the **functional level**, between the public version of PGP (version 8.0.2), the current trial version of PGP (now supplied by Symantec, but with most functions disabled, unless you use a valid license ☺), GnuPG, or even GPG4Win, although their interfaces are very different; the tutorial part of this document was designed using PGP 10.2.0 running on an Windows box, but it is perfectly possible (and even advisable) to use GnuPG. In this case you must take care to adapt the instructions to the GnuPG interface.
   The **free OpenPGP implementations** are available through this link. The trial **PGP** version can be obtained in the Symantec web site – we are interested in the "PGP Desktop

[1] Pretty Good Privacy (PGP) was created by Philip Zimmermann and was first published on the Internet in 1991. After several evolutions PGP was eventually acquired by Symantec. It is no longer an open source product, but several side projects continue to support the implementation of the same protocols and techniques, notably two initiatives in particular: OpenPGP and GnuPG.

Email" (designation my change); the limitations of the trial version will not compromise the homework execution.
**Kleopatra** is an (excellent) example of a certificate manager that meets all job requirements and can use instead of PGP.
**GnuPG** is available at http://www.gnupg.org/
The **GPG4Win** is available at http://www.gpg4win.org/

3. Along this document and by consistency reasons, the concept of "key" and associated "certificate" are not clearly differentiated, since this is the understanding of the PGP documentation. However, when dealing with X.509, the difference is relevant, as we will see.
4. The symbol 📄 denotes assertions that can be part of a security policy and are not under evaluation.

## Before you begin

1. Each user must have at least a **key-pair** (that will be the first task you will be required to do, later). This pair of keys consists of a **public key** (publicly available) and a **private key** (carefully saved). Within OpenPGP solutions, in your computer there are two important files, designated by ***keyrings***. One of them – `pubring.pkr` – stores all the public keys of users to which you want to send messages securely; the other – `secring.skr` – stores the private key(s). These files are stored in an encrypted file, in the user's workspace, (**... \<user folder>\PGP,** on Windows). But be aware that other applications may implement this storage differently, and it is expected that in all of them there will be an import/export mechanism, which allows you to easily transfer a key pair from one to another.

> 📄 Using more than one key pair is justified when you want to use more than one signature - we may want to use a personal signature and another institutional signature, with only some common identification elements.

2. When generating a key pair, in fact you may be associating multiple keys: a private key ("Master") for signing only; a second key, or more precisely a **sub key**, to encrypt; and one or more additional sub keys, to sign, encrypt or sign/encrypt, which can be revoked individually without compromising the "Master" key.

> 📄 This allows, for example, to maintain some signatures valid for an long time and change encryption sub key regularly (perhaps a period of one year). This can be a very useful security practice.

3. *Additional Decryption Keys* (ADKs) are additional keys, allowing those responsible for the security of an organization to decipher messages encrypted by the related public key.

> 📄 In principle, these keys will only be used in cases of extreme necessity! Otherwise it can create an important vulnerability.

4. ***Corporate Signing Key*** is a public key assigned to an organization and in which all users (some way related with the organization) can trust.

> 📄 All keys signed by the corresponding ***Corporate Key*** (private key) can be assumed as valid – while those not signed must be undertaken with extreme caution.

5. Key Validation: when you import someone else's public key, you can add it to your public keyring (or equivalent resource). But before you can use it to encrypt any message, you must carry out its **validation** (verify if the identification included in the public key corresponds in fact to the person with whom we want to interact):
    a. If the key was delivered personally, you can assume it is valid;
    b. If it was delivered by email or obtained from a server, then:
        i. If it is *signed* by someone you trust, then you can assume it is valid;

ii. If not, contact the key owner and ask him/her the public key's "fingerprint" which you must compare with the one enclosed in the imported key, as shown in Figure 1. If the verification succeeds the key is valid;

c. The result of the validation process can now be inserted in the target public key and sent back to the database (if you have got it from a server), to give notice of your opinion to other users;

d. If you have doubts about the validation you shall label the public key as "Marginally Valid" and, in case of non-validation, you should label it as "invalid".

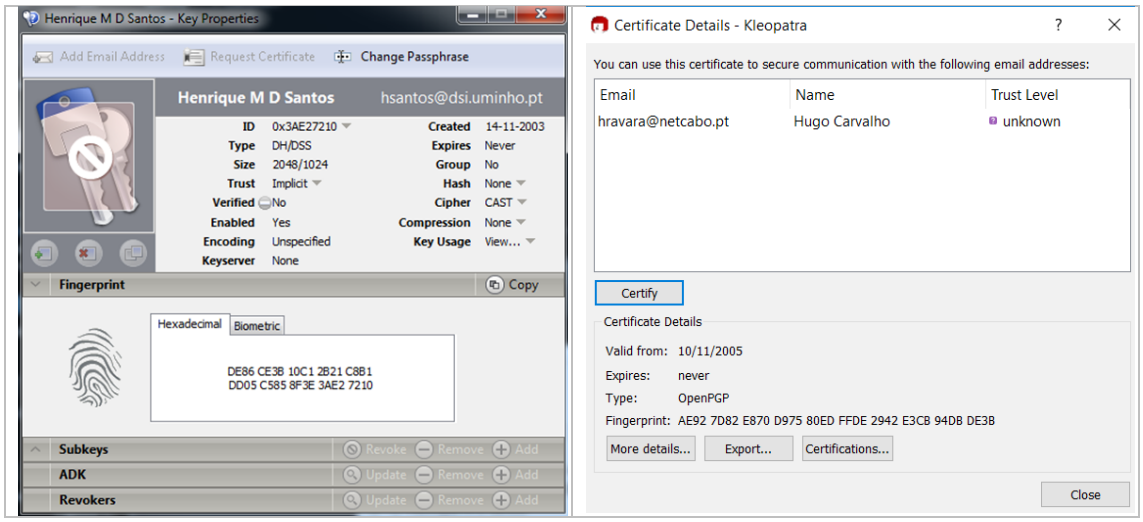



*Figure 1 - Checking the fingerprint in the key's properties window, using PGP (left side) and Kleopatra (right side)*

6. When using PGP, all operations on keys must be executed from the utility **PGP Desktop** – more detailed information about this utility can be obtained in the (excellent!) PGP help.
7. OpenSSL (https://www.openssl.org/) is a well-known and widely used open source project that includes also a cryptographic library (like GnuPG). However, OpenPGP and OpenSSL follow different standards and are not direct compatible (namely, PGP certificates, and X.509 certificates). And there are different opinions concerning performance of both solutions!

## Homework objectives

1. Understand how the concept of public key is typically implemented.
2. Recognize the operations associated with the management of public and private keys.
3. Develop skills in using digital certificate management tools.
4. Use a cryptographic framework to securely send and receive e-mail messages.

# Exercises

## Key management

### PGP option (at least one student should take it)

1. Run the PGP Desktop and follow the installation procedures listed (only for the first time you run it). As mentioned, you can alternatively use OpenSSL, or the Kleopatra certificate manager (on Linux or Windows, in this case included in the GP4Win project) - which allows you to simultaneously manage PGP and x509 certificates, but with some restrictions ☺.
2. Create a new key pair (`File-> New PGP key ...`). If this is the first time you run PGP, the creation of a new key-pair wizard should appear immediately and it is not necessary to use the menu.
3. When defining the parameters of your key pair, the option `Advanced` allows you to choose the type of algorithm and the key size. Choose the RSA key type with 1024/2048 bits. Do not select any expiration date and keep the original cipher algorithms list, keeping AES as the preferred one.
4. Next you will be asked to type a **Passphrase**, which you will need to enter whenever using your private key (to sign or decrypt local files). As you type, the quality of the pass phrase is sketched in a bar. Choose an easy to remember passphrase but not neglecting its quality.
5. Then you may be given the option to publish your public key on a server (only when creating a PGP certificate) at the same time that a check is made on your e-mail address. For now skip this check, since we will have the opportunity to publish the key later.
6. Back to the main window, select the key you have just created. This key is linked to a self-signature, which is essential to be able to export your key (see Figure 2). Check the properties of the signature and the key, paying particular attention to the fingerprint associated to the key – visualize it in hex mode and in text mode. Copy all the attributes of the key, the fingerprint and the signature to your logbook. Is this key your private key or the public key? Which elements link the signature to your private key?

Note: many people include a copy of this fingerprint in their business card, so that anyone who holds that card can validate the public key (or invalidate a false copy ☺).

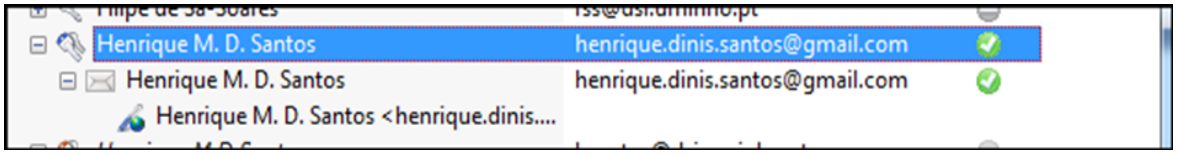



*Figure 2 - key and corresponding self-signature*

7. Still in the window that shows the properties of the key, select the tab `Subkeys`[2]. You can verify that your key has two sub keys, one for encryption and another for signature. However, as was described above, you can create more sub keys (Figure 3 illustrates a situation in which there are two sub keys, with different sizes and functions). Create one or

[2] It may be useful to look for more information concerning sub keys (a good starting point is http://mareichelt.de/pub/notmine/subkeys.html)

more sub keys. These sub keys are public or private keys? What is the relation of those keys with the master key? What is the usefulness of having multiple sub keys associated with a master key?

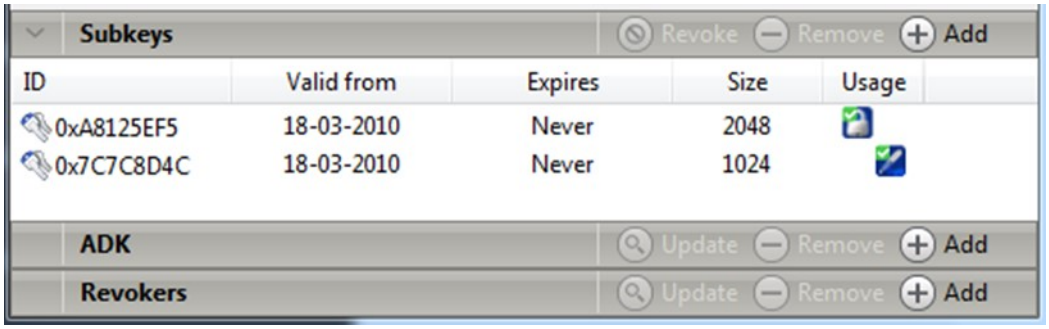



*Figure 3 - Key with two sub keys*

8. Now you have the possibility to request a certificate to a CA (Certification Authority), with this key pair, allowing you to obtain a X.509 certificate - in the same window that shows the key properties, there is a control button that allows you to generate the request, adding additional information to the key pair. If the application that you are using has this function, try it but it is not necessary to complete the request.
9. The next step is to export your public key. To do this you must first configure a PGP Server (this operation can be done in several ways and here we illustrate only one). From the `Tools` menu select the submenu `Edit Keyservers...`. You can verify that, by default, PGP is already configured to use a global repository supported by the organization that provides the OpenPGP (the repository is at keyserver.pgp.com, using the LDAP protocol). Nevertheless, you should add a new server that uses the Protocol **PGP Keyserver HTTP** - the server name is pgpkeys.mit.edu using the port 11371 (from that server you can directly search public keys, using a browser).
10. Back to the PGP Desktop utility, select your key and, via context menu (right-click) select `Send to > http://pgp.mit.edu` which will send your key to the configured server (don't worry, because PGP only sends the public key 😊). Now, through the menu `Tools` or the menu in the left pane, select the function `Search for Keys`, which lets you do a search on the server – you will have the opportunity to find a public key from someone you know and sign it, as a good user of the *web of trust* you just joined – do you still remember the meaning of this concept and its importance to the PGP community? 😊 Using the Web site referred in step 8, search for public keys by means of an email address (hsantos@dsi.uminho.pt) and then by name (Henrique Santos). Comment the obtained results.
11. An alternative way to deliver your public key to someone is to send it by email. To do this you can use the "drag-and-drop" technique from the PGP Key Manager window, directly to an email message. PGP is perfectly integrated with various email clients and enables you to send and receive public keys as messages attachments or as blocks of encrypted messages properly framed by specific PGP tags. When you receive such a message, you are faced with the option to automatically import the public key to the keyring – you need to exchange your key with your group members; do that using email and describe the process used. Should you do something more after receiving and storing your colleague's key? Note: don't forget the possibility of signing the message, so that the receiver can validate your public key by verifying the signature on the message!
12. You are ready to securely exchange authentic email messages with other PGP users and protect, through encryption and digital signature, critical information that you have on your computer.

Before finishing, it might be a good idea to export your public and private keys 😊. In the first case just to promote the secure (and valid) communication of public keys, in the second case... just for precaution.

The PGP Desktop allows you to perform many other operations that were not required in the context of this exercise. However, after assimilating the principle of operation of PKIs, and in particular of PGP, it will not be difficult to fully exploit the potential of this tool. For obvious reasons, one of the commands that has not been exercised and which has a very important role for the coherence of the web of trust is the `Keys-> Revoke ...`. Another concept that can be very useful is that of "User Groups".

## Option X.509 (at least one student should take it)

1. Install the OpenSSL (by default, most well-known Ubuntu releases, or related ones, already have this software package installed). You may use any other alternative software that has the ability to generate a public and private key pair (such as Kleopatra, referred above) but in the rest of this exercise it is assumed that you are using OpenSSL. If you use an alternative, you must adapt the commands/actions for your environment.
2. Create a new key pair (each student must create a key pair). If you use OpenSSL and want to get an RSA key pair, you must run the command (or some variant)
   `openssl genrsa -out privkey.pem 2048`
   which will create a private key and the associated 2048-bit public key, both stored in the same file, of type PEM[3]. The key thus obtained is suitable for encryption and signing and not requiring a password to be used, which in the context of certificate generation, to be handled by servers, is a good option - you can get more information in the OpenSSL documentation[4].
   Check the state of your private key with the command
   `openssl rsa -in privkey.pem -check`
   and record the command response, which includes the "text" with your new private key.
3. In order to integrate into a PKI, you should then prepare a file with a certificate request. This request will include your public key, associated with the previously generated private key, some personal and organizational information (mostly optional) and some attributes, including the Comman Name (CN) and email address (aspects of particularly important identification in the digital world), which will also be included in your certificate. This file will be sent to the **Certificate Authority (CA)**, which will return the certificate signed by its private key, after validating your identity (supposedly).
   In OpenSSL you can generate the certificate request using the command:
   `openssl req -new -key privkey.pem -out cert.csr`
   (adjusting file names as necessary), which will generate a request in **PKCS#10** format, a standard that most CAs accept.
   Check the status of your certificate request file with the command
   `openssl req -text -noout -verify -in cert.csr`
   and record the command response, which includes the attributes within your request and the future certificate.
4. OpenSSL also allows you to generate a self-signed certificate from your private key. That is useful in a scenario similar to the one promoted by the PGP web of trust, where it is not necessary (nor desirable) to have a top-level entity signing a certificate, but it is also the alternative to a CA's top certificate! Besides, a self-signed certificate is also often required

[3] In synthesis, PEM files contain ASCII encoded binary information, which facilitates copy / paste operations using simple text processing tools.

[4] https://www.openssl.org/docs/HOWTO/keys.txt

to import a private key into a particular environment / application. To generate the self-signed certificate you can use the command

```
openssl x509 -req -in cert.csr -signkey privkey.pem -out privcert.crt
```

The certificate thus obtained will be valid for one year (using the default OpnSSL configuration file), but the `days` option can be used to create certificates with different longevity).

Check the status of your self-signed certificate file with the command

```
openssl x509 -text -in privcert.crt
```

and record the command response; try to identify the elements you consider most relevant.

Steps 2 through 4 must be performed by each of the group members so that all have a key pair and a certificate request in their possession.

5. The next step aims to request the public certificate, duly signed by a CA. In this case, you will use a **dummy CA** specifically prepared for the exercise. OpenSSL includes all the tools required to create such a CA, and you are strongly encouraged to embrace such task.

> There are some useful tutorials explaining in detailed how to use OpenSSL to deploy a simple PKI, like the one available at this link. Being focused on simplicity, that proposal misses one relevant detail, which is the support of the certificate revocation check operation through OCSP[5]. However, it is also possible to find simple descriptions of how to implement that service (still using OpenSSL), such as the one available at this link. As a second alternative, we can choose a (more) ready-to-use platform, like XCA, or OpenCA, or even an interesting scheme to run a CA within a container. Yet another alternative consists of using a more robust and enterprise-oriented solution, like EJBCA, or the emergent smallstep project. Finally, there are also commercial solutions, some of them with trial versions.
> Despite the apparent simplicity, real PKIs are much more complex, requiring dedicated hardware for key generation (**HSM** - Hardware Security Modules), fault-tolerant distributed systems to assure continuous (as much as possible) operation and scalability, among other properties.

In case you are not interested in promoting technical skills on PKI development and management, you can access a very simple CA via the link https://hdsca.mafica.xyz/ (which provides an implementation of the first alternative solution described above). Through the interface, you can: 1) submit your certificate request (**Signing Service**), allowing to obtain your signed certificate; 2) download the **public CA certificate**, that you will need later to verify public certificates generated by the CA; and 3) revoke certificates (**Revoking Service**). Please note that the files returned to you have no extension and you should give them the `.crt` extension (although not required, it is advisable for easier identification).

Also note that you CANNOT SUBMIT THE SAME REQUEST twice (a certificate ID must always be unique and sourced from a unique ID request, too). Of course, the operation described above must be repeated for each of the group elements, for

[5] OCSP (Online Certificate Status Protocol) is an alternative to the use of the traditional CRL (Certification Revocation List) mechanism. With the later, a client download a list of revoked certificates and performs a check itself, while with the former the client submit the certificate ID to an OCSP server, which returns its status.

the respective X.509 certificates. If (hopefully) you decide to build your own PKI, do not forget to include a description of the work done in your logbook.

6. Back to OpenSSL and because to import your private key into different applications you will most likely need a PKCS#12 format file, you should execute the command:
`openssl pkcs12 -export -in pubcert.crt -inkey privkey.pem -certfile CAcert.crt -name "my-name" -out priv-pkcs12.p12`
where:
- `pubcert.crt` is your public certificate, signed by the CA - you may have to convert it first from binary to text encoding (you can verify by opening the file with any text editor); if necessary use the command
`openssl x509 -inform der -in cert.cer -out cert.pem`
- `privkey.pem` is your private key
- `CAcert.crt` is the CA public certificate (it may also need to be converted)
When executing the command to get the PKCS#12 file you will be asked for a password, which will serve to authenticate you when importing the private key and whenever you need to use it (it is not necessary to emphasize the importance of this password!). Of course, each group member has to repeat this process in order to import his / her private key.
Check the state of your file with the signed certificate and private key using the command
`openssl pkcs12 -info -in priv-pkcs12.p12`
and record the command response; try to identify the elements you consider most relevant. Compare the result with that obtained in step 4 and mark the differences.

📝 Before you finish, and as suggested for PGP, it may be a good idea to back up your public and private keys 😊.

## Send and receive secure messages

In this exercise we will use as a reference the Thunderbird e-mail client (with the Enigmail add-on already installed), including tasks with both PGP and X.509 certificates. However, thanks to the plugin integration method, the steps described apply to many other clients (unless the screen captures shown here 😊), such as Windows Live Mail, Eudora, or eM Client – in some cases you may have some trouble with the validation of a private certificate (X.509).

1. The first step aims to import PGP and X509 certificates into your platform/application. In the case of Thunderbird, the application itself includes the import function of both types of certificates:
   - X.509 certificates are imported through a manager accessible from the account setup (in Windows environment via the menu `Tools` → `Account Settings` → `Security`; in Linux environment via the menu `Edit` → `Account Settings` → `Security`);
   - PGP certificates are imported through the menu `Enigmail` → `Key Management`.
Other applications use the repositories of the OS itself, or one dedicated of the specific OpenPGP implementation in use. You should consult the respective documentation, without forgetting any validation procedures. Record in your logbook all the steps taken to install certificates on all computers in the group elements, referring to the environment used - the report should include a section per group element.
2. For X509 certificates, you must tell the email client which certificates you want to use for some specific operations (you may have more than one). From the menu `Tools`

(Windows) or `Edit` (Linux) choose the option `Account Settings` → `Security` (for the email account you are using). From here you can:

a. Manage the certificates you have on your computer with the option `Manage Certificates` (see Figure 4); you can see your own certificates (typically public and private keys), those of other people, servers, recognized CAs, and others; You can also import certificates into any of the previous classes - which you will have already done in the previous step. Since at this stage you are importing X.509 certificate which are signed by a CA, the respective public certificate must also be imported. Check and record in your logbook the evidences showing that.

b. Choose the certificate you want to use for signing, and encrypting and decrypting (although they may differ in most cases is the same). Register in the loogbook an image that documents the configuration performed.

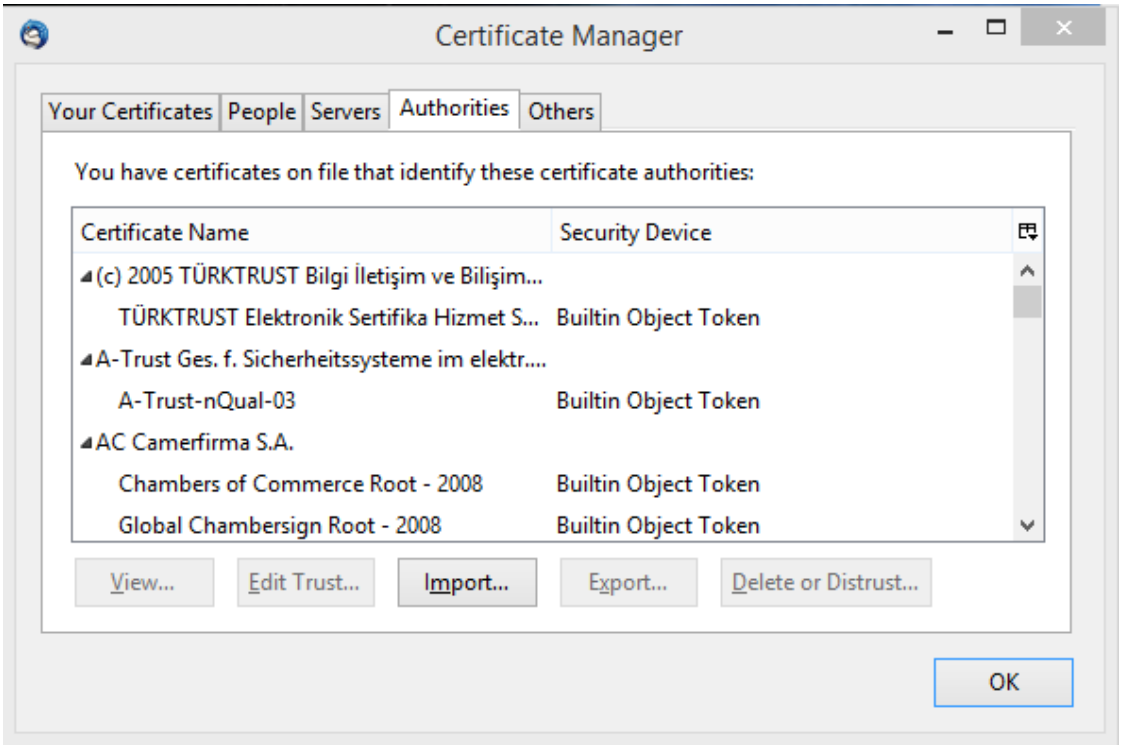



*Figure 4- Thunderbird's Certificate Manager Window*

3. For PGP certificates, the equivalent operation is performed through the `Enigmail` menu by selecting `Preferences` and then `Display Expert Settings and Menus`. The window that appears will give you access to several functions that you can explore later, but for now we will only refer to the `Key Selection` tab. In this tab you should select the first three options (see Figure 5), which will allow the application to select the proper key using the email address as the primary identifier, only requiring manual intervention if it is not possible to infer which certificate to use. It is also important to highlight the ability to create specific rules for specific email addresses (`Edit Rules` button), which allows for an interesting degree of flexibility in managing how Enigmail responds to encrypted/signed messages, depending on the sender and the recipient.

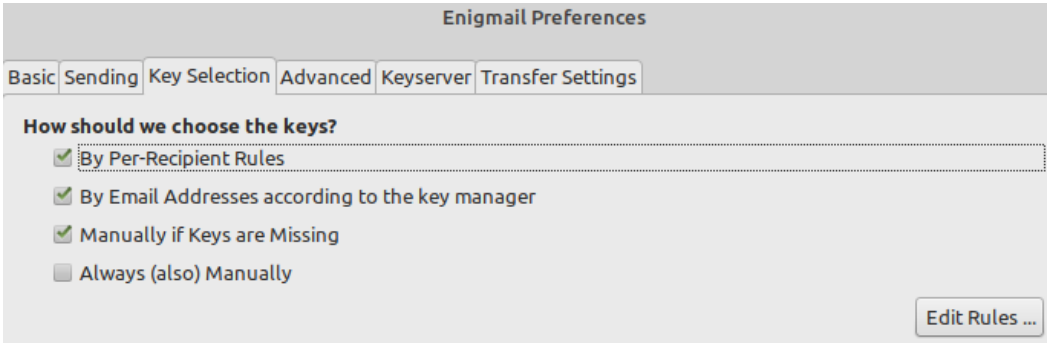



*Figure 5 – Enigmail window to configure how PGP keys are selected*

Note that (whether you use PGP certificates or X509 certificates):

a. The email client usually uses the email address to choose the certificates, if your signing certificate has a different email address than the one you use to send email, you may not be able to sign messages!

b. If someone sends you a public certificate by email, it should be automatically saved; for X509 certificates, this only happens if the CA is recognized - in your case this will not happen, because the CA you use is fictitious and not properly registered; but you can effectively "force" your system to recognize and accept your CA by simply uploading its public certificate into the Root Authorities category.

c. The use of webmail does not usually allow this type of operation, assuming that this is done on the personal computer at the file level and using some specific software for this purpose. Examples include the iSafeguard ™ security suite, the Google Chrome FlowCrypt extension, and GPG. To make digital signatures Adobe Reader fits perfectly, as does HelloSign (a web application that integrates well with the Google environment)

All the email clients of all group members must be properly configured. After that, you should exercise the message exchange, with signature and cipher. You should document all experiments in the logbook, with the results, and each element should be expected to send and receive at least one message to and from all other elements.

4. The next exercise aims to revoke one of the certificates and check the effect.

| |
|---|
| This operation is not reversible, so be careful what you do. |

The way to revoke a certificate and the expected result is different in the two models – PGP, with the Web of Trust model, with a shared central repository and no centralized management; and X509, with a well-structured hierarchy, based on a top CA.
(i) In the first case (**PGP**), revocation consists in issuing a revocation certificate signed by the private key – in case of loss of the private key, there is no way to revoke ☹, which poses a threat to the consistency of the process.

| |
|---|
| For that reason, it is good practice to produce a revocation certificate at the moment you create the key pair, storing that revocation certificate carefully (possibly on the same backup as the private key). |

The revocation certificate must be sent to the server, so that future downloaders will know its status. Most PGP servers exchange information periodically, so this revoked certificate will eventually spread, but there is no automatic mechanism for clients to update themselves (Kleopatra includes a refresh function `Tools` → `Refresh OpenPGP Certificates` that allows you to update all PGP

certificates in pubkeyring, but of course getting information from the server it is configured with - Figure 6 illustrates the registration of a revoked certificate).
(ii) In the second case (**X.509**) the process is different as there is a top centralized entity responsible for these aspects of certificate management. In this case there are two mechanisms available: **Certification Revocation Lists (CRL)** and **Online Certificate Status Protocol (OCSP)**.

a. **CRL** - as the name implies, it is a list maintained and signed by the CA, with all the revoked key IDs. How often this list is updated depends on the CA policy, but in any way, it is the client's responsibility to download the list and check the state of each certificate it keeps locally stored. Most certificate management programs allow you to configure this function automatically. Certificates issued by a CA usually (but not necessarily) include a URL indicating where the list can be obtained – **CDP (CRL Distribution Point)**. Except for being centralized, comparing to the PGP model this mechanism highlights the same limitations concerning the update response time.

b. **OCSP** – on its turn is an online service designed to provide a certificate's usage status immediately. CAs implementing this service allow for a more efficient time response, only showing limitations when the user is offline. However, it is possible to implement and maintain both mechanisms, which complement each other in the advantages/limitations.
The certificates you obtained from the CA in the initial phase of this exercise support OCSP by enabling the required attribute on the produced certificates (check the **Authority Info Access** attribute - sometimes referred to only by **authInfo** - of your certificate (s) X.509).

Revoke at least two of the certificates (one PGP and one X.509) and verify the impact of the operation on the message exchange process. Describes the logbook experience, taking care to clearly indicate any changes and verifications you have made.

| | | | | | | |
|---|---|---|---|---|---|---|
| **Henrique M. D. Santos** | **henrique.dinis.santos@gmail.com** | **certified** | **18/03/2010** | | **OpenPGP** | **2D75 B09D CA...** |
| ~~**Henrique M D Santos**~~ | ~~**hsantos@dsi.uminho.pt**~~ | ~~**not certified**~~ | ~~**14/11/2003**~~ | | ~~**OpenPGP**~~ | ~~**C585 8F3E 3AE...**~~ |
| Hans Hedbom | hans.hedbom@kau.se | not certified | 04/09/2014 | 03/09/2019 | OpenPGP | C6B5 5146 E1A8 ... |
| Filipe de Sa-Soares | fss@dsi.uminho.pt | not certified | 18/11/2002 | | OpenPGP | C94C 781B 6206 ... |
| Charles Heselton | charles-heselton@cox.net | not certified | 11/01/2004 | | OpenPGP | 7BF8 D1F6 4829 ... |

*Figure 6 – Example of the PGP certificate revocation state provided by Kleopatra*

## Protect lcal documents

1. Most certificate management tools allow some additional operations, such as encrypting files or folders. PGP and Kleopatra are no exception. In the case of PGP these operations can be performed directly from the PGP Desktop utility (or PGP tools, depending on which version you have installed) using the PGP Zip function group, more precisely the PGP Zip Assistant - see Figure 7. Through a simple Drag and drop operation you can encrypt and/or signed various files and folders. Kleopatra includes the equivalent functions in the `File` menu (see Figure 7).
2. The same set of commands is available from the so-called Windows and Linux context menus. In any window, select one or more files and press the right mouse button. The context menu that appears gives you direct access to the cipher and/or signature functions, as shown in .
3. There are other very useful operations in the PGP Desktop, such as those associated with the `PGP Disk` menu. These operations let you create a new encrypted virtual disk,

encrypt a complete disk, or permanently erase the contents of a disk's free space.
Try some of these features, not forgetting to document all experiences.

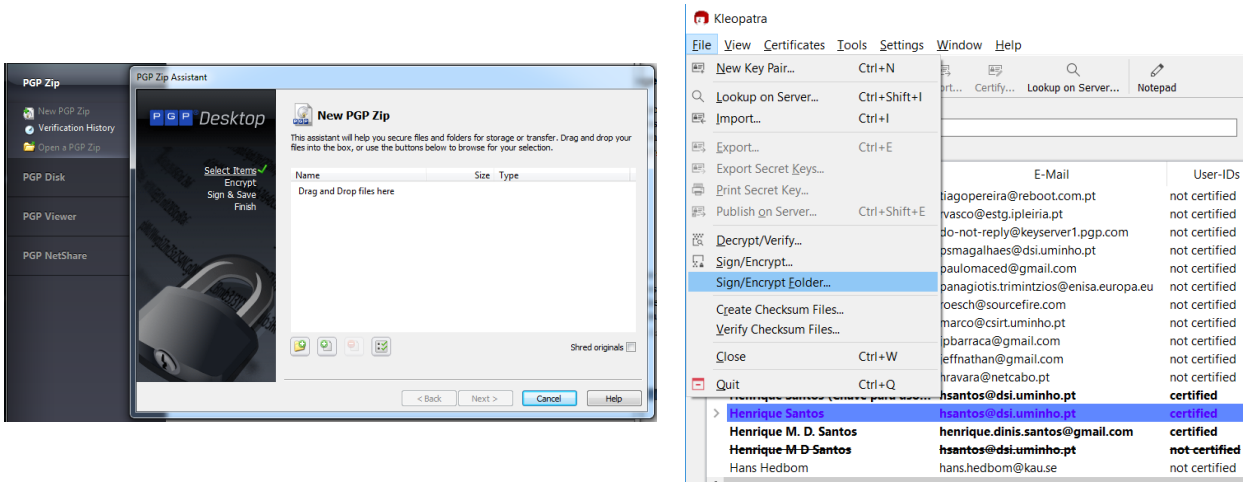
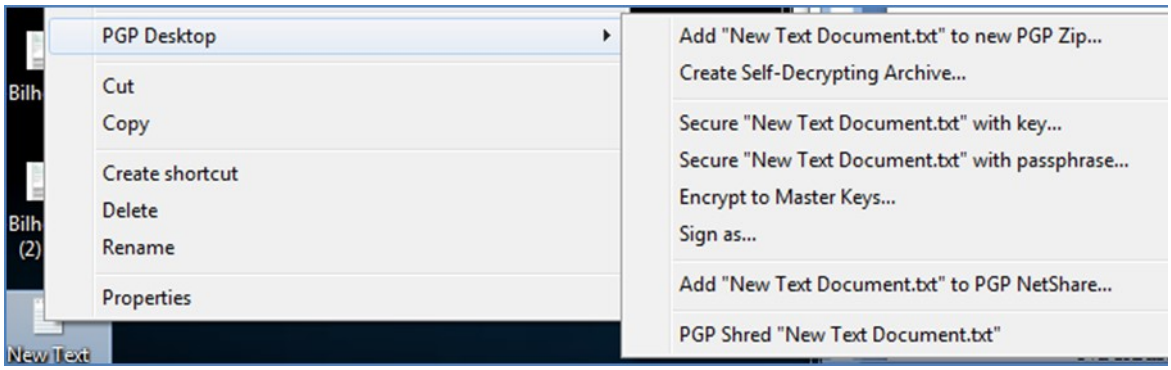



*Figure 7 - File/folder encryption with PGP (left) and Kleopatra (right)*



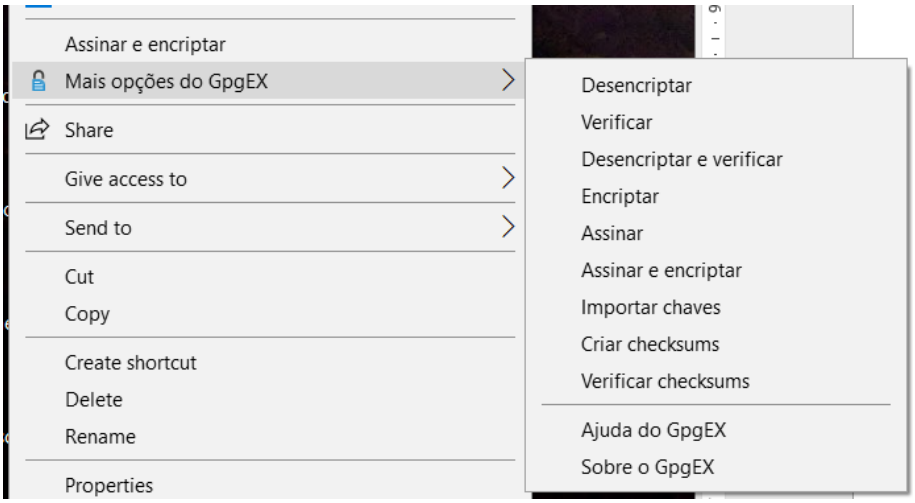




*Figure 8 - Access to cryptographic functions from the context menu: PGP above; and Kleopatra below*